Expense Tracker with Spending Insights

Project Plan, Technical Design and Development Roles

Industry Readiness Programme I - End Term I

Course	B.Tech CSE, 2nd Year, Group 3G
Language / Stack	Python, SQLite, Pandas, Matplotlib, Streamlit, pytest
Team size	4 members: 2 backend, 1 frontend, 1 integration and QA
Industry	Fintech, banking apps, personal finance management
Marks	100, scaled to 50
Presentations	Began 5 November 2026

1. Project Overview

The Expense Tracker lets a user record expenses, assign categories and budgets, and then turns that data into insights: where the money goes, how spending changes by month, which categories are over budget, which expenses look unusual, and what the month is likely to total. Everything is stored in SQLite and shown in a Streamlit dashboard.

Problem statement

People often cannot see where their money leaks. A plain list of expenses does not show patterns, overspending or unusual transactions. Fintech apps solve this with automatic categorisation, budgets and alerts; this project builds a working version of that pipeline.

Objectives

- Store expenses reliably with validation (no negative amounts, valid dates, known categories).
- Set monthly budgets per category and show budget vs actual with alerts.
- Produce insights: category breakdown, monthly trend, top spends, anomalies, month-end forecast.
- Provide an interactive Streamlit dashboard plus a CLI fallback.
- Support CSV import and export.

Features and concepts to explain

Feature	Concept to explain
Add, edit, delete expenses	CRUD operations, SQLite, SQL basics
Categories and payment mode	Data modelling, constraints
Monthly budget with alerts	Business rules, thresholds (OK, Warning, Over)
Category breakdown and monthly trend	Pandas groupby and aggregation
Unusual spend detection	Z-score computed within each category
Month-end forecast	Run-rate projection (spent so far / days elapsed x days in month)
CSV import / export	File handling, validation of external data
Dashboard	Streamlit widgets, Matplotlib charts

2. Technology Stack and Justification

Tool	Layer	Why chosen
Python	All	Fast development, strong data libraries, industry standard in fintech and data work
SQLite	Backend	Zero setup, built into Python, enough for a single user; PostgreSQL when scaling
Pandas	Backend	Groupby, resampling and date handling for insights
Matplotlib	Frontend	Pie, bar and line charts with full control
Streamlit	Frontend	Interactive forms and dashboards in pure Python
pytest	QA	Simple, standard testing
Git / GitHub	DevOps	Parallel work with branches and pull requests

Safety tip: generate 3-6 months of realistic sample data (script included in database.py) so the demo always has meaningful charts, even on a fresh laptop.

3. Database Design

Table	Column	Type	Notes
expenses	id	INTEGER, primary key	Auto-increment
expenses	date	TEXT (YYYY-MM-DD)	Required; converted to datetime in Pandas
expenses	amount	REAL	Required; CHECK amount > 0
expenses	category	TEXT	Required; from the category list in config.py
expenses	note	TEXT	Optional description
expenses	payment_mode	TEXT	UPI, Card, Cash, Net Banking
budgets	category	TEXT, primary key	One row per category
budgets	monthly_limit	REAL	Required; greater than 0

4. Architecture and File Structure

Three layers: the **Data layer** (SQLite and validation), the **Analytics engine** (insights from DataFrames) and the **Frontend** (Streamlit and charts). A CLI entry point sits beside the UI as a fallback. Each file has one job and one owner.

Layer	File	Purpose	Owner
Backend: data	config.py	Categories, payment modes, default budgets, thresholds	Member 1
Backend: data	database.py	SQLite setup, CRUD, validation, CSV import/export, sample data	Member 1
Backend: analytics	insights.py	Summary, trend, budget status, anomalies, forecast, top spends	Member 2
Frontend	app.py	Streamlit dashboard and add-expense form	Member 3
Frontend	charts.py	Pie, bar and line charts	Member 3
Integration / QA	main.py	CLI fallback entry point	Member 4
Integration / QA	tests/	Unit tests for validation and insights	Member 4
Integration / QA	requirements.txt, README.md	Dependencies, setup and run guide	Member 4

Data flow: app.py form -> database.py (validate and save) -> get_expenses() DataFrame -> insights.py -> charts.py -> app.py display.

5. Core Code (Starter Version)

config.py and database.py (Member 1)

```
# config.py
CATEGORIES = ["Food", "Transport", "Rent", "Shopping", "Bills", "Health", "Entertainment", "Other"]
PAYMENT_MODES = ["UPI", "Card", "Cash", "Net Banking"]
DEFAULT_BUDGETS = {"Food": 6000, "Transport": 2500, "Shopping": 3000, "Entertainment": 2000}
WARNING_PCT = 80          # budget usage above this shows a warning
ANOMALY_Z = 2             # z-score above this is flagged as unusual

# database.py
import sqlite3, random
import pandas as pd
from datetime import date, timedelta
from config import CATEGORIES, PAYMENT_MODES

DB = "expenses.db"

def conn():
    return sqlite3.connect(DB)

def init_db():
    with conn() as c:
        c.execute("CREATE TABLE IF NOT EXISTS expenses (id INTEGER PRIMARY KEY AUTOINCREMENT, "
                  "date TEXT NOT NULL, amount REAL NOT NULL CHECK(amount > 0), "
                  "category TEXT NOT NULL, note TEXT, payment_mode TEXT)")
        c.execute("CREATE TABLE IF NOT EXISTS budgets (category TEXT PRIMARY KEY, "
                  "monthly_limit REAL NOT NULL CHECK(monthly_limit > 0))")

def add_expense(day, amount, category, note="", mode="UPI"):
    if amount <= 0:
        raise ValueError("Amount must be positive")
    if category not in CATEGORIES:
        raise ValueError(f"Unknown category: {category}")
    day = pd.to_datetime(day).strftime("%Y-%m-%d")      # raises on a bad date
```

```
    with conn() as c:
        c.execute("INSERT INTO expenses (date, amount, category, note, payment_mode) VALUES (?,?,?,?,?)",
                  (day, float(amount), category, note, mode))

def get_expenses(start=None, end=None):
    with conn() as c:
        df = pd.read_sql_query("SELECT * FROM expenses ORDER BY date", c)
    df["date"] = pd.to_datetime(df["date"])
    if start is not None:
        df = df[df["date"] >= pd.to_datetime(start)]
    if end is not None:
        df = df[df["date"] <= pd.to_datetime(end)]
    return df

def delete_expense(expense_id):
    with conn() as c:
        c.execute("DELETE FROM expenses WHERE id = ?", (expense_id,))

def set_budget(category, limit):
    with conn() as c:
        c.execute("INSERT OR REPLACE INTO budgets VALUES (?, ?)", (category, float(limit)))

def get_budgets():
    with conn() as c:
        return dict(c.execute("SELECT category, monthly_limit FROM budgets").fetchall())

def import_csv(path):                                    # columns: date, amount, category, note, payment_mode
    for _, r in pd.read_csv(path).iterrows():
        add_expense(r["date"], r["amount"], r["category"], r.get("note", ""), r.get("payment_mode", "UPI"))

def seed_sample_data(days=120):
    random.seed(42)
    ranges = {"Food": (80, 450), "Transport": (30, 300), "Shopping": (300, 3000),
              "Bills": (200, 1500), "Entertainment": (150, 1200), "Health": (100, 1500)}
    today = date.today()
    for i in range(days):
        d = today - timedelta(days=i)
        for cat in random.sample(list(ranges), k=random.randint(1, 3)):
            lo, hi = ranges[cat]
            add_expense(d, random.randint(lo, hi), cat, "sample", random.choice(PAYMENT_MODES))
        if d.day == 1:
            add_expense(d, 9000, "Rent", "monthly rent", "Net Banking")
```

insights.py (Member 2)

```
import pandas as pd
from config import WARNING_PCT, ANOMALY_Z

def category_summary(df):
    return df.groupby("category")["amount"].sum().sort_values(ascending=False)

def monthly_trend(df):
    return df.groupby(df["date"].dt.to_period("M").astype(str))["amount"].sum()

def top_spends(df, n=5):
    return df.nlargest(n, "amount")[["date", "category", "amount", "note"]]

def current_month(df):
    return df[df["date"].dt.to_period("M") == df["date"].max().to_period("M")]

def budget_status(df, budgets):
    spent = current_month(df).groupby("category")["amount"].sum()
    rows = []
    for cat, limit in budgets.items():
        s = float(spent.get(cat, 0))
        pct = round(s / limit * 100, 1)
        status = "Over" if s > limit else ("Warning" if pct >= WARNING_PCT else "OK")
        rows.append({"category": cat, "spent": s, "limit": limit, "used_pct": pct, "status": status})
    return pd.DataFrame(rows)

def flag_anomalies(df):
```

```
    g = df.groupby("category")["amount"]
    z = (df["amount"] - g.transform("mean")) / g.transform("std")
    return df[z > ANOMALY_Z]

def forecast_month_end(df):
    today = df["date"].max()
    month = current_month(df)
    return month["amount"].sum() / today.day * today.days_in_month
```

charts.py (Member 3)

```
import matplotlib.pyplot as plt
from insights import category_summary, monthly_trend

def pie_by_category(df):
    s = category_summary(df)
    fig, ax = plt.subplots(figsize=(6, 6))
    ax.pie(s.values, labels=s.index, autopct="%1.0f%%", startangle=90)
    ax.set_title("Spending by category")
    return fig

def bar_monthly(df):
    t = monthly_trend(df)
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.bar(t.index, t.values)
    ax.set_title("Monthly spending")
    ax.set_ylabel("Amount (INR)")
    plt.xticks(rotation=45)
    return fig
```

app.py - Streamlit frontend (Member 3)

```
import streamlit as st
from datetime import date
from config import CATEGORIES, PAYMENT_MODES
from database import init_db, add_expense, get_expenses, get_budgets
from insights import (category_summary, budget_status, flag_anomalies,
                      forecast_month_end, top_spends, current_month)
from charts import pie_by_category, bar_monthly

st.set_page_config(page_title="Expense Tracker", layout="wide")
init_db()
st.title("Expense Tracker with Spending Insights")

with st.sidebar.form("add_form"):
    st.header("Add expense")
    d = st.date_input("Date", date.today())
    amount = st.number_input("Amount (INR)", min_value=0.0, step=50.0)
    category = st.selectbox("Category", CATEGORIES)
    note = st.text_input("Note")
    mode = st.selectbox("Payment mode", PAYMENT_MODES)
    if st.form_submit_button("Save"):
        try:
            add_expense(d, amount, category, note, mode)
            st.success("Saved")
        except ValueError as e:
            st.error(str(e))

df = get_expenses()
if df.empty:
    st.info("No expenses yet. Add one from the sidebar.")
    st.stop()

c1, c2, c3 = st.columns(3)
c1.metric("Total spent", f"{df['amount'].sum():,.0f}")
c2.metric("This month", f"{current_month(df)['amount'].sum():,.0f}")
c3.metric("Forecast (month end)", f"{forecast_month_end(df):,.0f}")

st.subheader("Budget status")
for _, r in budget_status(df, get_budgets()).iterrows():
    st.write(f"{r['category']}: {r['spent']:,.0f} / {r['limit']:,.0f} ({r['status']})")
```

```
    st.progress(min(r["used_pct"] / 100, 1.0))

left, right = st.columns(2)
left.pyplot(pie_by_category(df))
right.pyplot(bar_monthly(df))
st.subheader("Unusual expenses"); st.dataframe(flag_anomalies(df))
st.subheader("Top spends"); st.dataframe(top_spends(df))
st.download_button("Download CSV", df.to_csv(index=False).encode(), file_name="expenses.csv")
```

main.py and tests (Member 4)

```
# main.py  (CLI fallback)
from database import init_db, add_expense, get_expenses, get_budgets
from insights import category_summary, budget_status, forecast_month_end

init_db()
print(category_summary(get_expenses()))
print(budget_status(get_expenses(), get_budgets()))
print("Forecast:", round(forecast_month_end(get_expenses()), 2))

# tests/test_basic.py
import pandas as pd, pytest
from database import add_expense
from insights import category_summary, forecast_month_end

def test_negative_amount_rejected():
    with pytest.raises(ValueError):
        add_expense("2026-11-01", -50, "Food")

def test_unknown_category_rejected():
    with pytest.raises(ValueError):
        add_expense("2026-11-01", 50, "Gold")

def test_category_summary():
    df = pd.DataFrame({"date": pd.to_datetime(["2026-11-01", "2026-11-02"]),
                       "amount": [100, 50], "category": ["Food", "Food"]})
    assert category_summary(df)["Food"] == 150

def test_forecast():
    df = pd.DataFrame({"date": pd.to_datetime(["2026-11-05"]), "amount": [500], "category": ["Food"]})
    assert forecast_month_end(df) == 500 / 5 * 30
```

Run the app with `streamlit run app.py`. Call `seed_sample_data()` once to fill the database for the demo. These are working starters; final versions need comments, edit/delete in the UI and more tests.

6. Development Roles (4 Members)

Roles are split by development area: two backend members, one frontend member, and one for integration, QA and deployment. Each member owns specific files and does not edit others' files.

Member	Area	Owns	Tasks
Member 1	Backend: Data layer	database.py, config.py	SQLite setup, CRUD, validation (no negative amounts, valid dates, known categories), budgets table, CSV import/export, sample data generator
Member 2	Backend: Analytics engine	insights.py	Category summary, monthly trend, budget vs actual, anomaly detection, forecast, top spends
Member 3	Frontend	app.py, charts.py	Add-expense form, filters, pie/bar/line charts, budget progress bars, alerts, metric cards, CSV download
Member 4	Integration, QA and DevOps	main.py, tests/, requirements.txt, README.md	Git repo and branches, CLI fallback, unit tests, error handling, merging code, deployment (Streamlit Cloud), final bug fixing

Contract between members (agree on day 1)

Function	Owner	Output
add_expense(day, amount, category, note, mode)	Member 1	Saves a row or raises ValueError
get_expenses(start, end)	Member 1	DataFrame: id, date (datetime), amount, category, note, payment_mode
get_budgets()	Member 1	Dict of category to monthly limit
category_summary(df)	Member 2	Total per category (Series)
monthly_trend(df)	Member 2	Total per month (Series)
budget_status(df, budgets)	Member 2	DataFrame: category, spent, limit, used_pct, status
flag_anomalies(df)	Member 2	Rows with z-score above the threshold
forecast_month_end(df)	Member 2	Projected month total (number)
pie_by_category(df), bar_monthly(df)	Member 3	Matplotlib figures

Data flow between people: Member 1 (data) -> Member 2 (insights) -> Member 3 (UI and charts) -> Member 4 (integrates, tests, deploys).

Git rules to avoid conflicts

- One branch per member: data, analytics, frontend, integration. Only Member 4 merges into main.
- Nobody edits another member's file; request a change instead.
- Member 3 starts immediately with dummy DataFrames; Member 2 starts with a hand-made sample DataFrame.
- Commit small and often with clear messages; pull main before starting each day.
- Cross-reviews: Members 2 and 3 swap, Members 1 and 4 swap, so everyone can explain another part in Q&A.;

7. Development Timeline (3 Days)

Day	Member 1 (Data)	Member 2 (Analytics)	Member 3 (Frontend)	Member 4 (QA / Integration)
Day 1	DB and CRUD, sample data	Summary, trend, top spends	Layout with dummy data	Repo, branches, requirements, agree contract
Day 2	Validation, budgets, CSV import	Budget status, anomalies	Connect real functions, charts	Unit tests, CLI, merge branches
Day 3	Edge cases, error messages	Forecast, review	Polish, alerts, download	Deploy, README, full test run, final fixes
Before slot	Test on demo laptop	Test on demo laptop	Test on demo laptop	Record backup demo video

If time is short, drop optional items (edit expense, line chart, deployment) and keep the core: add expense, categories, budget status, summary chart, forecast.

8. Real-World Industry Usage

Aspect	Details
Sector	Fintech and banking apps (for example Walnut, Fi, Jupiter, bank mobile apps)
Problem solved	Users cannot see where money leaks; apps need categories, budgets and alerts without manual effort
Real workflow	Transactions arrive from UPI, cards or bank SMS, get categorised, are compared to budgets, and trigger alerts or insights
How to scale	Move SQLite to PostgreSQL, add user accounts and authentication, schedule monthly reports
Integration	Expose the analytics engine as a REST API (FastAPI); auto-categorise UPI and SMS transactions; connect to bank APIs
Limitations	Manual entry, no bank sync, simple run-rate forecast, single user

9. Presentation Plan (10 Minutes)

All four members must speak. Each presents the area they built.

Time	Section	Speaker
0:00 - 1:30	Problem, objective, industry relevance, architecture overview	Member 4
1:30 - 3:00	Data layer: database design, validation, sample data	Member 1
3:00 - 5:00	Analytics: summary, budget status, z-score anomalies, forecast	Member 2
5:00 - 8:30	Live Streamlit demo: add expense, see budget alerts, charts, anomalies	Member 3
8:30 - 10:00	Testing, challenges, future scope, conclusion	Member 4

Slide outline (8 to 10 slides, clean and technical)

- 1. Title and team 2. Problem and industry 3. Objectives
- 4. Architecture diagram 5. Database design 6. Insights explained (z-score, forecast)
- 7. Demo (dashboard screenshot) 8. Testing results 9. Challenges and future scope 10. Conclusion

Demo checklist

- Laptop charged, app already running, database seeded with sample data.
- Rehearsed flow: add one large expense live, then show it appear as a budget alert and an anomaly.
- Screen recording ready in case the demo fails; screenshots saved as a last resort.
- Be clear that the forecast is a simple projection, not machine learning.

10. Likely Questions and Answers

Question	Short answer
Why SQLite?	Zero setup, built into Python, enough for a single user; PostgreSQL when scaling.
Why Streamlit?	Builds an interactive UI in pure Python with no web-development overhead.
How do you detect unusual spend?	Z-score within each category: how many standard deviations above that category's average. Flag above 2.
How is the forecast made?	Spent so far divided by days elapsed, times days in the month. Simple, not ML.
How do you prevent bad data?	Validation in add_expense and CHECK constraints in SQLite: positive amounts, valid dates, known categories.
What is a SQL injection and how do you avoid it?	Malicious input inside SQL; we use parameterised queries (? placeholders).
How do budgets work?	Monthly limit per category; status is OK, Warning at 80% or more, Over above 100%.
Limitations?	Manual entry, no bank sync, simple forecast, no login.
How would it scale?	PostgreSQL, user accounts, REST API, auto-categorisation of UPI/SMS data.
Future scope?	ML categorisation, receipt scanning, bank integration, savings goals, multi-user.

11. Report Outline (Max 10 Pages, Soft Copy)

Section	Content	Length	Owner
1. Problem and objective	Problem, goals, why it matters	0.5 page	Member 4
2. Scope and requirements	What it does and does not do, inputs and outputs	0.5 page	Member 4
3. Tech stack	Tools and justification table	0.75 page	Member 4
4. Implementation	Architecture diagram, database design, analytics, frontend, key code	2.5 pages	Members 1, 2, 3
5. Testing and results	Test case table, dashboard screenshots, insight examples	2 pages	Members 4 and 2
6. Industry application	Sector, workflow, scaling, integration	1 page	Member 3
7. Challenges and learnings	Real problems and how you solved them	0.75 page	All members
8. Conclusion and future scope	Summary, limitations, next steps	0.75 page	Member 4

Formatting: title page with team names and roles, headings, page numbers, captioned screenshots, consistent font. Export to PDF and check it is under 10 pages. Member 4 merges all sections.

Suggested test cases

Test	Input	Expected result
Valid expense	2026-11-01, 250, Food	Saved and shown in the list
Negative amount	-50	Rejected with a clear message
Zero amount	0	Rejected
Invalid date	31-31-2026	Rejected, no crash
Unknown category	Gold	Rejected
Empty database	Fresh install	Friendly message, no crash
Over budget	Spend above the limit	Status shows Over and the bar is full
Anomaly	One very large expense	Appears in the unusual expenses table
CSV import	File with a bad row	Bad row reported, valid rows saved

12. How to Score Well

Weak version	Strong version
Just add and list expenses	Budgets with alerts, anomaly flags, forecast
No tests	Tests for validation and insights
Empty or fake-looking data	3-6 months of realistic sample data
Members only know their file	Everyone can explain their module and one other

Bucket (25 each)	What earns top marks
Technical	Clean, commented code; everyone can explain SQL, groupby and z-score; working Streamlit app
Report	All 8 sections, screenshots and test table, under 10 pages, clean formatting
Presentation and industry	Smooth live demo, clear sector, problem and scaling story
Soft skills	All 4 speak, good timing, calm and logical Q&A;, professional conduct

Upgrades if time permits

- Edit and delete expenses from the UI.
- Savings goals with progress tracking.
- Simple keyword-based auto-categorisation from the note field.
- FastAPI endpoint exposing the analytics engine.

Final checklist

- [ ] Confirm slot date and report deadline with the CR / coordinator
- [ ] Group Lead nominated and informed (guideline deadline was 26.10.26, confirm status)
- [ ] App runs end to end on the demo laptop, database seeded, backup recording ready
- [ ] Report exported as PDF, under 10 pages
- [ ] Speaking parts split among all 4 members
- [ ] Two timed rehearsals completed
- [ ] Code is original and commented; each member can explain their own module and one other